



中山大學
SUN YAT-SEN UNIVERSITY

本科生课程报告

软件配置与运维文档

寻迹校园失物招领平台

课程名称： 软件工程

项目名称： 寻迹校园失物招领平台

团队名称： 寻迹项目组

项目负责人： 朱梓涵

团队成员： 朱梓涵、周星辰、林洪宽、郭焜华、程文韬

负责人学号： 24325356

文档版本： V1.0

基线日期： 2026 年 7 月 12 日

中山大学

2026 年 7 月

目录

1	文档目的与范围	1
2	配置管理	1
3	版本控制	4
4	持续集成	5
5	部署架构	7
6	部署流程	8
7	运维计划	9
8	备份、恢复与回滚	11
9	账号与运行安全	12
10	结论	13

1 文档目的与范围

本文规定「寻迹」校园失物招领平台的配置管理、版本控制、持续集成、部署和运维方法。文档的目标是让开发成员使用一致的配置名称和版本规则，让部署人员能够按固定步骤启动系统，并让运行期间的检查、备份和变更都有明确流程。

表 1: 配置与运维管理范围

管理方向	采用的机制	管理目的
配置管理	环境变量、示例模板、Pydantic Settings、Vite 构建参数	分离代码、环境差异和敏感值
版本控制	Git 分支、规范化提交、版本标签、Alembic revision	追踪应用与数据库的每次变化
持续集成	GitHub Actions 并行执行静态检查、测试和构建	在合并前使用统一规则检查各模块
部署管理	Dockerfile、Compose、健康检查和部署清单	以相同步骤构建和启动六个服务
运行维护	日志、运行指标、巡检、备份、恢复和回滚流程	保持服务与数据处于可管理状态

2 配置管理

2.1 配置分层

项目遵循「代码与配置分离、敏感值与模板分离、不同环境使用相同变量名」的原则。配置按以下四层组织：

1. **代码默认值**：只保存通用且非敏感的运行参数，例如 API 前缀、默认页大小和本地端口。
2. **示例模板**：列出部署所需变量、说明和示例格式，空出密码、token 和私钥。
3. **环境配置**：部署者从模板生成本地环境文件，为当前环境填写真实值；环境文件不进入版本库，也不复制到镜像层。

4. **启动注入**：Compose 将环境变量传入容器，Backend 和 AI Service 通过 Pydantic Settings 读取；前端 API 基址通过 Vite 构建参数写入静态产物。

进程环境变量优先于环境文件中的同名值，便于部署平台在不改动仓库文件的情况下覆盖配置。配置名称保持大写下划线形式，布尔值使用 true/false，列表值采用英文逗号分隔，时间参数在名称中明确使用秒、分钟或小时。

2.2 配置项分类

表 2:主要配置类别

类别	代表变量	管理规则
数据库	DB_HOST、DB_PORT、DB_NAME、DB_USER、DB_PASSWORD	库名和账号按环境区分，密码由部署环境注入
身份鉴权	JWT_SECRET_KEY、JWT_EXPIRE_DAYS	共享环境使用独立随机密钥，不沿用示例值
管理员初始化	BOOTSTRAP_ADMIN_ENABLED、管理员账号、密码和手机号	仅首次初始化时开启，账号建立后关闭开关
对象存储	MinIO endpoint、access key、secret key、bucket、签名时长	bucket 保持私有，浏览器和 AI 使用不同有效期的签名
AI 服务	AI_SERVICE_BASE_URL、AI_SERVICE_TOKEN、timeout	主后端和 AI 服务配置同一随机 token，服务间调用携带鉴权头
模型供应商	API key、base URL、文本模型、向量模型和视觉模型	key 由部署环境保管，模型名按环境切换
Web 访问	CORS_ALLOWED_ORIGINS、VITE_API_BASE_URL	后端使用显式来源列表，前端构建参数与部署域名一致
后台任务	匹配间隔、任务轮询、最大尝试次数、退避和锁超时	时间单位写入变量名，修改时同步说明和运维记录
本地演示	DEBUG、SMS_DEBUG_ENABLED、SMS_DEMO_PHONES	调试验证码仅对明确手机号列表生效，共享环境关闭调试开关

2.3 启动校验

Compose 对 MySQL root 密码、业务数据库账号、JWT 密钥、管理员账号和 MinIO 凭据使用必填变量表达式，缺少关键值时停止配置展开。Backend 在非 Debug 环境拒绝示例 JWT 密钥和示例管理员密码，并拒绝通配 CORS；AI Service 要求服务 token

至少 32 个字符，只有显式本地模式允许绕过。启动校验把配置规则放到程序入口，避免依赖部署者记忆。

2.4 环境划分

表 3: 环境配置策略

环境	用途	配置特点	数据策略
本地开发	单人开发与调试	本机端口、可开启精确短信白名单、独立密钥	每名成员使用独立测试数据
团队联调	跨模块接口联调	固定 API 地址、统一 CORS、关闭调试返回值	使用团队测试账号与可重建数据
课程演示	答辩和功能展示	冻结版本、固定端口、独立随机密钥	预置演示数据，部署前制作快照
正式部署	长期提供服务	HTTPS、最小端口暴露、独立服务账号	执行周期备份和恢复校验

环境之间复用变量名称和部署结构，但不复用数据库、对象 bucket 或访问凭据。配置文件只保存当前环境需要的值；从联调环境切换到演示环境时，通过替换环境文件和镜像版本完成，不在源代码中加入环境判断。

2.5 配置变更流程

1. 提交者说明变量名称、类型、默认行为、取值范围和适用环境。
2. 同步更新 Settings、Compose 映射和示例模板；前端变量同时更新构建说明。
3. 本地执行配置展开与应用启动检查，确认缺省值和必填值行为。
4. 通过 Pull Request 评审变量命名、敏感性和兼容性。
5. 部署时先在联调环境应用新配置，再按相同名称更新演示或正式环境。
6. 记录变更时间、应用版本、变量名称和操作者，不记录秘密值本身。

密码、JWT 密钥、AI token、数据库口令和对象存储 secret 不写入 issue、提交信息、日志或截图。需要更换凭据时，先生成新值并更新服务端，再更新调用方，确认新值生效后撤销旧值。

3 版本控制

3.1 Git 分支与提交

仓库以 `main` 保存稳定交付版本，以 `develop` 汇集团队增量；功能和修复分别使用 `feature/<name>`、`fix/<name>`。每个 Pull Request 聚焦一个功能或一个问题，并关联需求编号或任务。提交信息采用简短的 Conventional Commit，例如：

```
feat(FR-02): add lost item publish api
fix(FR-05): correct claim review transition
test(match): cover fallback calculation
docs: update deployment instructions
chore(ci): adjust compose check
```

表 4: 版本控制对象与标识

对象	版本标识	管理方法
源代码与文档	Git commit SHA	同一提交保存相关代码、迁移、配置模板和说明
稳定版本	Git tag	标签指向经评审的固定提交，不移动已有标签
Python 依赖	uv.lock	Backend 与 AI Service 分别冻结解析版本
前端依赖	pnpm-lock.yaml	用户端和管理端分别执行冻结安装
数据库结构	Alembic revision	每次结构变化形成有顺序的迁移版本
容器镜像	应用名加 commit SHA 或版本号	部署记录能够反查源码与依赖

`.gitattributes` 统一普通文本为 LF，Windows 命令脚本使用 CRLF，二进制文件不参与行尾转换。`.gitignore` 排除环境文件、依赖目录、缓存、日志和构建中间产物，最终课程 PDF 作为交付文件单独纳入版本控制。

3.2 数据库版本

数据库使用 SQLAlchemy 模型描述当前结构，使用 Alembic 保存从旧版本到新版本的变化。迁移文件按单一 revision 链排列，Backend 容器在启动 Uvicorn 前运行

alembic upgrade head。手写 Schema 文件不作为新环境初始化入口，避免 ORM 与独立建表脚本产生两套结构定义。

表 5: 数据库变更管理规则

变更类型	迁移要求	评审内容
新增表或字段	明确类型、nullability、默认值和索引	ORM 与数据库设计是否一致
新增唯一约束	先整理历史重复记录，再建立约束	数据转换顺序和业务唯一性
修改状态值	给出旧值到新值的映射	Service 状态机与枚举是否同步
删除字段	先停止代码读写，再迁移数据并删除	新旧应用是否存在兼容窗口
历史数据库接入	备份并逐项比较表、列、索引和约束	结构等价后才允许记录当前 revision

迁移提交同时修改 ORM、数据库说明和相关测试。自动生成的迁移只作为初稿，提交者需要检查数据语句和 downgrade 行为。部署记录保存变更前后 revision，便于确认环境所处版本。

4 持续集成

4.1 触发与并行任务

GitHub Actions 在 main、develop 的 push 和 Pull Request 上运行。相同分支出现新提交时，旧运行被取消；Backend、AI Service、User App、Admin Web 和 Docker Build 五类任务并行执行。所有任务采用仓库锁文件安装依赖，因而开发者可以在本地复现同一组命令。

表 6: 持续集成任务

任务	执行步骤	管理作用
Backend	uv 安装 ; Ruff lint/format ; Mypy ; Pytest 与覆盖率	检查主后端规范、类型和业务逻辑
AI Service	uv 安装 ; Ruff lint/format ; Mypy ; Pytest	检查独立服务及其 HTTP 契约
User App	pnpm 冻结安装 ; vue-tsc ; Vitest ; Vite build	检查用户端类型、逻辑和生产构建
Admin Web	pnpm 冻结安装 ; vue-tsc ; Vitest ; Vite build	检查管理端类型、逻辑和生产构建
Docker Build	Compose config ; Backend 与 AI 镜像构建	检查编排配置和镜像构建入口

流水线失败时，提交者根据任务日志在原分支修正，并由新提交触发重新检查。覆盖率 XML 作为 Backend 任务产物保存；Compose 配置检查使用专用占位值，只验证环境变量映射和编排语法，不读取部署环境中的真实凭据。

4.2 版本进入部署的条件

进入演示或正式部署的版本应指向固定 commit 或 tag，并满足：Pull Request 已评审；持续集成任务成功；环境模板与新增变量同步；数据库迁移随版本提交；部署者能够从锁文件构建镜像。部署记录使用 commit SHA、镜像标识、Alembic revision 和配置版本共同描述一次发布，避免只记录「最新版」。

5 部署架构

5.1 六服务拓扑

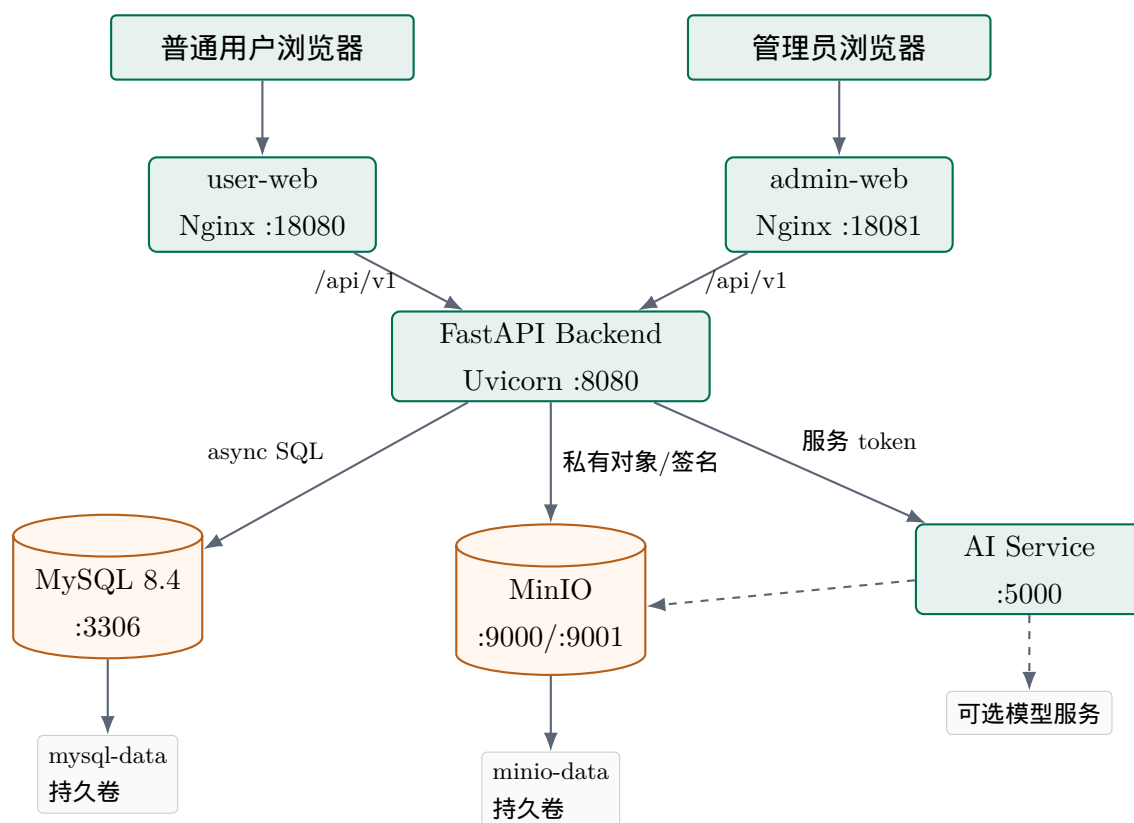


图 1: 完整栈部署拓扑

两个前端由 Nginx 提供静态文件，并将 `/api/v1` 转发到 Backend。Backend 管理业务、数据库迁移和对象签名；AI Service 作为独立 HTTP 服务运行，通过 service token 鉴权；MySQL 和 MinIO 使用命名卷保存数据。数据库、对象存储和 AI 端口在本地配置中绑定回环地址，外部用户只访问前端和 Backend 入口。

5.2 服务清单

表 7: Compose 服务与启动关系

服务	镜像或构建	默认端口	数据与健康检查	启动关系
mysql	MySQL 8.4	127.0.0.1:3306	mysql-data ; mysqladmin ping	先启动并达到 healthy

表 7 (续)

服务	镜像或构建	默认端口	数据与健康检查	启动关系
minio	固定日期镜像	127.0.0.1: 9000/9001	minio-data	先于 Backend 启动
ai-service	项目 Dockerfile	127.0.0.1:5000	HTTP health	先于 Backend 启动
backend	项目 Dockerfile	8080	HTTP health ; 启动前迁移	等待 MySQL healthy
user-web	Node 构建、Nginx 运行	18080	无状态静态服务	Backend 启动后提供访问
admin-web	Node 构建、Nginx 运行	18081	无状态静态服务	Backend 启动后提供访问

所有服务采用 `restart: unless-stopped`。完整栈配置用于团队联调和课程演示；前端独立配置用于只构建两个 Web 应用；Podman host 配置用于特定 rootless 网络环境；正式部署叠加端口收敛配置，取消 MySQL 和 MinIO 的主机端口映射。

6 部署流程

6.1 部署前准备

部署者确认主机具备 Docker Compose 或 Podman Compose、可用端口和足够磁盘空间，然后从示例模板生成环境文件。所有空白密码和 token 填写独立随机值，确认前端 API 地址、CORS 来源、MinIO 公共签名地址和 AI 服务地址与目标环境一致。数据库部署还需确认当前 Alembic revision，并在结构变更前制作备份。

完整栈启动命令

```
cp deploy/docker/.env.example deploy/docker/.env
# Fill required values in deploy/docker/.env.
docker compose --env-file deploy/docker/.env \
    -f deploy/docker/docker-compose.yml config --quiet
docker compose --env-file deploy/docker/.env \
    -f deploy/docker/docker-compose.yml up --build -d
docker compose --env-file deploy/docker/.env \
    -f deploy/docker/docker-compose.yml ps
```

先执行 `config --quiet` 检查变量展开和 YAML 合并，再构建并后台启动服务。Backend 容器先运行 `alembic upgrade head`，迁移成功后才启动 Uvicorn。首次初始化管理员时显式开启 `bootstrap`，账号建立并确认可登录后关闭该开关。

6.2 部署后核验

1. 查看六个服务的状态与启动日志，确认没有重复重启。
2. 访问 Backend 和 AI Service 的 `/health`。
3. 打开用户端和管理端，检查静态资源及 `/api/v1` 代理。
4. 检查数据库 revision、MinIO bucket 私有策略和对象签名访问。
5. 使用演示账号执行登录、发布、搜索、匹配、认领、交接和审核的最小走查。
6. 登记部署时间、commit SHA、镜像标识、数据库 revision、配置版本和执行人。

6.3 发布变更顺序

仅修改无状态应用时，先构建固定版本镜像，再逐个替换 AI、Backend 和前端服务；数据库结构变化时，按照「备份 → 迁移 → Backend → 前端 → 走查」的顺序执行。新增字段优先采用兼容方式，使旧应用和新应用在替换期间都能读取；删除字段先停止应用读写，再在后续版本移除。

7 运维计划

7.1 职责与周期

表 8: 运行维护计划

周期	负责人	检查内容	记录内容
每次部署	发布负责人、后端负责人	服务状态、健康接口、revision、主流程走查	版本、配置、迁移和核验结果
每日	当值成员	服务存活、错误日志、任务状态、磁盘与对象容量	异常时间、requestId 和处理动作
每周	运维负责人	数据库容量、慢查询、任务积压、日志空间和备份清单	趋势、维护项和负责人

表 8 (续)

周期	负责人	检查内容	记录内容
每月	项目负责人、运维负责人	账号权限、凭据有效期、依赖版本和恢复抽查	变更决定和下次检查时间
重大变更前	开发负责人、发布负责人	数据备份、迁移演练、版本兼容和回滚入口	操作顺序、执行人和确认人

7.2 健康、日志与指标

MySQL使用 `mysqladmin ping` , Backend 与 AI Service 提供 HTTP health , Compose 根据健康状态安排 Backend 启动。应用使用 Loguru 输出启动、请求异常、外部服务调用、后台任务重试和关键状态信息；管理员审核、举报处理和用户管理等操作写入业务操作日志。HTTP 异常响应携带 `requestId` , 便于将客户端反馈与服务日志关联。

表 9: 运行观察项目

对象	观察内容	处理方式
HTTP 服务	请求量、延迟、4xx/5xx、健康状态	按时间和 <code>requestId</code> 定位接口与版本
MySQL	连接数、慢查询、锁等待、存储空间	检查查询、索引、连接池和磁盘
MinIO	对象容量、上传错误、签名错误	核对 bucket、凭据、网络 and 对象引用
AI Service	调用延迟、超时、规则处理比例	检查模型配置、服务 token 和调用配额
后台任务	PENDING、RUNNING、FAILED 数量与最老任务时间	根据任务类型和重试记录处理
容器主机	CPU、内存、磁盘和容器重启次数	清理日志或镜像并调整资源配置

日志不记录密码、OTP、JWT、AI token、对象存储 `secret` 或证件原图。运行日志按日期轮转，演示和联调环境保留最近一周；业务操作日志随数据库备份保存。部署、配置变更和数据恢复使用独立运维记录，避免与应用日志混在一起。

7.3 后台任务维护

匹配、分类和敏感内容处理使用持久任务表。运维人员关注任务状态、尝试次数、下次执行时间和锁定时间；遗留的 RUNNING 任务按锁超时规则回收，失败任务保留最后错误摘要。手工重试前先确认输入对象仍存在、业务版本没有变化，并避免同一任务被重复提交。

8 备份、恢复与回滚

8.1 备份计划

表 10:数据与版本备份计划

对象	方法	周期与保留	校验方法
MySQL	一致性逻辑备份；长期环境同时保留 binlog	每日，保留 7 日；周备份保留 4 周	在隔离数据库中导入并检查 revision
MinIO	将 bucket 镜像到独立存储并保存对象清单	每日增量、每周全量	抽样校验 hash，并与数据库对象引用对账
配置	保存无秘密模板、变量清单和加密凭据副本	每次配置变更	在新环境中执行配置展开
镜像与源码	Git tag、commit SHA、锁文件和不可变镜像	每次发布	从版本记录取得对应镜像和迁移
运维记录	部署、迁移、备份、恢复和配置变更记录	随操作归档，按学期保存	检查时间、版本、执行人和核验项

备份副本与运行主机分开保存，敏感备份加密并限制访问。每次备份记录开始时间、结束时间、版本、大小和校验值。课程演示前制作数据库与对象存储快照，演示数据发生修改时可恢复到统一初始状态。

8.2 恢复流程

1. 确定目标应用版本、数据库 revision、对象清单和配置版本。
2. 在隔离环境创建空 MySQL 与空 MinIO，确认备份文件和解密凭据可用。

3. 恢复数据库和对象，核对数据库中的对象引用是否都能找到对应文件。
4. 使用与备份匹配的应用镜像启动，确认 revision 后按顺序迁移到目标 head。
5. 检查健康接口，并走查登录、发布、搜索、认领、交接和管理审核。
6. 由发布负责人确认后切换服务入口，保存恢复过程和核验结果。

8.3 应用与数据库回滚

应用回滚使用上一稳定 commit 对应的固定镜像，不重新构建同名本地镜像。前端静态资源、Backend 和 AI Service 使用同一发布记录中的兼容版本；切换后重新执行健康检查和最小业务走查。

数据库优先采用兼容迁移和向前修正。只增加字段的版本可以保持新旧应用兼容；涉及删除或数据改写时，部署前制作备份并在隔离副本演练。Alembic downgrade 只在确认不会丢失数据时使用，否则恢复部署前备份，再启动与该 revision 对应的应用版本。

9 账号与运行安全

表 11: 运行安全规则

领域	配置规则	运维动作
服务凭据	JWT、AI token、数据库和 MinIO 使用独立随机值	定期更换并确认旧值停止生效
网络入口	数据库、MinIO 和 AI 默认不向公网开放	正式环境只通过 HTTPS 网关提供 Web 入口
对象访问	bucket 私有，按角色生成短期签名	定期检查匿名策略和签名有效期
管理员账号	首次初始化后关闭 bootstrap	定期检查管理员列表和操作日志
调试功能	共享环境关闭 Debug 和短信调试返回	部署核验时检查开关与手机号白名单
容器镜像	按锁文件构建并使用固定版本标识	发布时记录镜像标识和源码 commit

凭据更换按照「生成新值、更新服务端、更新调用方、检查新值、撤销旧值」的顺序进行。数据库和 MinIO 为应用创建专用账号，管理员只在维护时使用管理账号。配置和日志的访问权限按角色分配，课程演示账号不具备修改基础设施配置的权限。

10 结论

本文给出了寻迹项目的配置与运维方法：配置通过环境变量和模板分层管理；源代码、依赖、数据库和镜像使用明确版本标识；持续集成统一执行静态检查、测试与构建；Compose负责六服务部署；部署清单、巡检、日志、指标、备份、恢复和回滚构成日常运维流程。

上述规则将「使用哪个版本、如何配置、怎样部署、运行后检查什么」转化为可重复的步骤，使开发、联调、课程演示和正式部署能够沿用同一套管理方式。